Zadanie nr 2 - sieć MADALINE do rozpoznawania znaków

Inteligentna Analiza Danych

Łukasz Centkowski 264303
Maciej Dominiak 264307

25 października 2026

1 Cel zadania

Zadanie polegało na zbudowaniu sieci MADALINE rozpoznającej znaki graficzne. Wzorce znaków sieć wczytuje z plików obrazowych. Sprawdziliśmy, do jakiego poziomu zaszumienia rozpoznawanie działa poprawnie i od czego ta granica zależy.

2 Wstęp teoretyczny

Neuron liniowy liczy iloczyn skalarny swoich wag i wejścia. Gdy oba wektory mają długość jeden, wynik jest cosinusem kąta między nimi, czyli liczbą z przedziału $[0, 1]$ mówiącą, jak bardzo są do siebie podobne [1].

Sieć MADALINE opiera się na tej własności. Ma jedną warstwę neuronów liniowych i każdy neuron odpowiada za jeden znak wzorcowy. Wszystkie neurony dostają to samo wejście, a wygrywa ten o największym wyjściu [2, 3].

Wagi neuronu ustawiamy po prostu na znormalizowany wzorzec jego znaku, więc *nie ma tu żadnego treningu*. Obraz zamieniamy na wektor w trzech krokach: binaryzacja, sklejenie wierszy jeden za drugim, podzielenie przez długość. Tusz koduje jedynka, tło zero. Przy odwrotnej konwencji wszystkie podobieństwa wyszłyby bliskie jedności, bo tło zajmuje ponad 90% obrazu.

Podobieństwo wzorca do jego zaszumionej wersji można przewidzieć. Niech wzorzec ma n pikseli tuszu, obraz P pikseli, a każdy piksel odwraca się z prawdopodobieństwem p. Wspólnych pikseli tuszu zostaje średnio $(1-p)\,n$, a z tła dochodzi $p\,(P-n)$ nowych. Stąd

$$\cos \;=\; \frac{(1-p)\,n}{\sqrt{n}\,\sqrt{(1-p)\,n + p\,(P-n)}}\,. \tag{1}$$

3 Eksperymenty i wyniki

Implementacja to dwa programy konsolowe. `font_generator.py` rysuje literę i zaszumia ją, a `madaline_ocr.py` buduje sieć ze wzorców w jednym katalogu i rozpoznaje znaki z drugiego. Plik `description.txt` wiąże nazwy obrazów z etykietami znaków. Użyliśmy liter `a`, `b` i `c` w kroju Times New Roman, na obrazach 128×128 pikseli.

3.1 Eksperyment nr 1: weryfikacja wobec przykładu z wykładu

3.1.1 Założenia

Materiały do wykładu podają gotowe wyniki dla kształtów 4×4 liter X, Y, Z oraz zaburzonej litery X. Można więc sprawdzić implementację na wartościach znanych z góry, bez udziału generowanych obrazów.

3.1.2 Przebieg

Trzem neuronom przypisaliśmy znormalizowane kształty liter X, Y i Z. Na wejście podaliśmy zaburzoną literę X, która różni się od wzorca jednym pikselem.

3.1.3 Rezultat

Porównanie zawiera Tab. 1.

Tabela 1: Stopnie podobieństwa dla kształtów 4×4 z materiałów wykładowych

Wzorzec oryginalny	**Materiały**	**Implementacja**
litera X	0,943	0,943
litera Y	0,816	0,816
litera Z	0,738	0,738

Wszystkie trzy wartości zgadzają się co do trzeciego miejsca po przecinku. Normalizacja wektorów i liczenie podobieństwa działają więc poprawnie.

3.2 Eksperyment nr 2: rozpoznawanie znaków zaszumionych

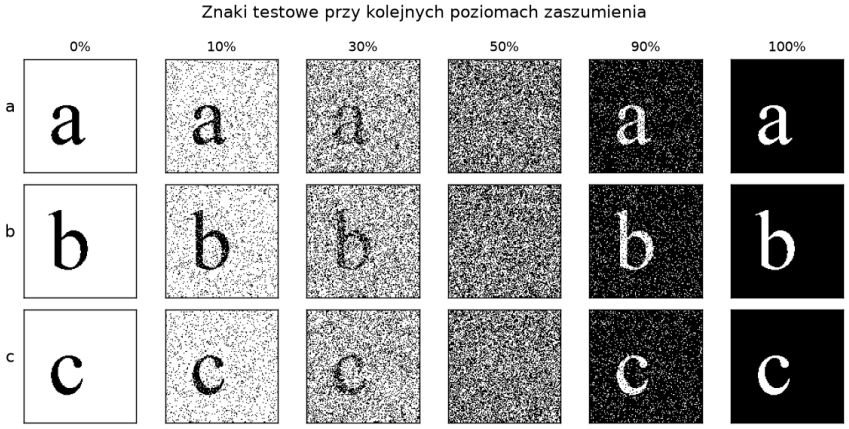
3.2.1 Założenia

Wzorce wygenerowaliśmy bez szumu, a znaki testowe przy sześciu poziomach: 0, 10, 30, 50, 90 i 100%. Poziom to prawdopodobieństwo odwrócenia

jednego piksela. Szum dotyczy zarówno tuszu, jak i tła. Przy 100% odwrócone są wszystkie piksele, więc znak testowy jest negatywem wzorca: biała litera na czarnym tle.

3.2.2 Przebieg

Sieć zbudowaliśmy ze wzorców bez szumu. Potem rozpoznawaliśmy kolejne zestawy znaków testowych i zapisywaliśmy wskazanie sieci oraz podobieństwo do zwycięskiego wzorca. Jak wyglądają znaki testowe, pokazuje Rys. 1.



Rysunek 1: Znaki testowe przy kolejnych poziomach zaszumienia

3.2.3 Rezultat

Wyniki zestawiono w Tab. 2; wytłuszczono wskazania błędne.

Do 30% zaszumienia sieć nie myli się ani razu. Przy 50% rozpoznaje poprawnie jeden znak na trzy, a przy 90% i 100% żadnego. Uwagę zwraca szybki spadek podobieństwa: przy 10% szumu wynosi ono już tylko około 0,5, choć człowiek nadal czyta literę bez trudu, co widać na Rys. 1.

Przy 50% i 90% sieć wskazuje niemal zawsze literę `b`, bez względu na to, co jest na obrazie.

Negatyw (100%) człowiek czyta bez trudu, a sieć myli się na wszystkich trzech znakach. Sieć porównuje piksele tuszu, a w negatywie tuszem jest dawne tło. Negatyw nie ma więc z własnym wzorcem ani jednego wspólnego piksela tuszu i neuron właściwej litery daje dokładnie 0. Wygrywa zawsze wzorzec innej litery.

Tabela 2: Rozpoznawanie znaków przy rosnącym zaszumieniu

Poziom szumu	**a** →	**b** →	**c** →	**Poprawnie**
0%	a (1,000)	b (1,000)	c (1,000)	3/3
10%	a (0,522)	b (0,577)	c (0,489)	3/3
30%	a (0,264)	b (0,303)	c (0,244)	3/3
50%	**b** (0,170)	b (0,174)	**b** (0,167)	1/3
90%	**b** (0,170)	**a** (0,128)	**b** (0,153)	0/3
100%	**b** (0,170)	**a** (0,126)	**b** (0,151)	0/3

3.3 Eksperyment nr 3: przyczyna spadku stopnia podobieństwa

3.3.1 Założenia

Chcieliśmy ustalić dwie rzeczy. Po pierwsze, czy szybki spadek podobieństwa to cecha metody, czy błąd w kodzie. Po drugie, dlaczego przy dużym szumie wygrywa zawsze ten sam neuron. Postawiliśmy hipotezę, że decyduje udział tuszu w powierzchni obrazu.

3.3.2 Przebieg

Policzyliśmy piksele tuszu każdego wzorca i porównaliśmy zmierzone wyjście neuronu danej litery z wartością ze wzoru (1). Porównujemy wyjście neuronu *właściwej* litery, a nie zwycięskiego, bo powyżej 30% szumu wygrywa już neuron innej litery.

3.3.3 Rezultat

Udział tuszu zestawia Tab. 3, a porównanie z modelem Tab. 4 oraz Rys. 2.

Tabela 3: Udział pikseli tuszu w powierzchni obrazu

Wzorzec	**Piksele tuszu**	**Piksele ogółem**	**Udział**
litera a	761	16384	4,6%
litera b	999	16384	6,1%
litera c	609	16384	3,7%

Tabela 4: Wyjście neuronu właściwej litery kontra model (1)

	Poziom zaszumienia					
Wzorzec	0%	10%	30%	50%	90%	100%
litera a, pomiar	1,000	0,522	0,264	0,138	0,026	0,000
litera a, model	1,000	0,524	0,267	0,152	0,023	0,000
litera b, pomiar	1,000	0,577	0,303	0,174	0,027	0,000
litera b, model	1,000	0,576	0,303	0,175	0,027	0,000
litera c, pomiar	1,000	0,489	0,244	0,130	0,016	0,000
litera c, model	1,000	0,482	0,241	0,136	0,021	0,000

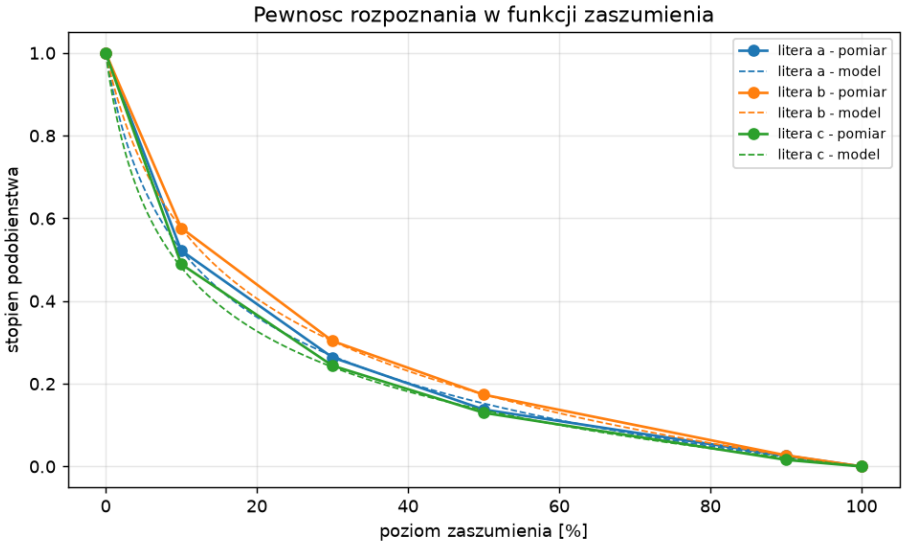
Model trafia w pomiar na całym zakresie, z różnicą najwyżej 0,015. Spadek podobieństwa jest więc cechą metody, a nie błędem w kodzie. Przyczynę widać we wzorze (1). Litera zajmuje kilka procent obrazu, więc przy 10% szumu z tła dochodzi około 1560 losowych pikseli tuszu, czyli *dwa razy więcej*, niż ma sama litera. Mianownik rośnie wtedy dużo szybciej, niż maleje licznik.

Tabela 3 tłumaczy też przewagę litery `b` przy dużym szumie. Dla wejścia bliskiego losowemu średni iloczyn skalarny z neuronem i rośnie jak $\sqrt{n_i}$, więc wygrywa neuron o najliczniejszym wzorcu. Litera `b` ma 999 pikseli tuszu, a pozostałe 761 i 609. Dlatego wygrywa niezależnie od tego, co jest na obrazie.

4 Wnioski

Poprawność i odporność na szum. Dla kształtów 4×4 z materiałów wykładowych nasza implementacja zwraca te same wartości co do trzeciego miejsca po przecinku. Wzorce bez szumu rozpoznaje z podobieństwem równym dokładnie 1,000. Rozpoznawanie jest bezbłędne do 30% zaszumienia. Przy 50% poprawny jest jeden znak na trzy, a przy 90% i 100% żaden. Negatywu znaku sieć nie rozpoznaje wcale, choć dla człowieka jest on czytelny.

Jak czytać stopień podobieństwa. To nie jest prawdopodobieństwo, tylko cosinus kąta. Spadek do około 0,5 przy 10% szumu nie znaczy, że sieć „prawie się myli”. Znak jest wtedy rozpoznawany pewnie, a przewaga nad



Rysunek 2: Zmierzony stopień podobieństwa i krzywa modelu (1)

pozostałymi wzorcami pozostaje wyraźna. Niska wartość bierze się stąd, że szum dotyka całego obrazu, a litera zajmuje kilka procent jego powierzchni. Skuteczność zależy więc nie tylko od poziomu szumu, ale też od proporcji tuszu do tła. W praktyce znaczy to tyle, że obraz warto kadrować ciasno wokół znaku.

Brak mechanizmu odrzucenia. Sieć MADALINE zawsze wskazuje jakiegoś zwycięzcę, nawet gdy na wejściu nie ma żadnego znaku. Przy dużym szumie wskazuje po prostu wzorzec o największej powierzchni, co pokazał eksperyment nr 3. Praktyczny system rozpoznawania znaków wymagałby progu odrzucenia opartego na stopniu podobieństwa. Dla badanego zestawu próg taki musi leżeć między 0,174, czyli najwyższą wartością towarzyszącą wskazaniu błędnemu, a 0,244, czyli najniższą towarzyszącą wskazaniu poprawnemu; wartość 0,2 oddziela te dwa zbiory w całym przebadanym zakresie zaszumienia.

5 Załączniki

Rdzeń implementacji: wektor wagowy każdego neuronu ustawiany wprost na znormalizowany wzorzec, bez treningu iteracyjnego (plik `madaline.py`).

```
network = Network([len(vectors[0]), len(vectors)], bias=False,
                  activations=[(linear, linear_derivative)])
for neuron, vector in zip(network.layer_list[0].neurons, vectors):
    neuron.weights = vector
```

Bibliografia

[1] K. Stokfiszewski. *Inteligentna Analiza Danych. Materiały do wykładu nr 1 i nr 2*. Instytut Informatyki Politechniki Łódzkiej.

[2] B. Widrow, M. E. Hoff. *Adaptive switching circuits*. IRE WESCON Convention Record, 1960.

[3] S. Osowski. *Sieci neuronowe do przetwarzania informacji*. Oficyna Wydawnicza Politechniki Warszawskiej, Warszawa.